Dưới đây là lời giải chi tiết và hoàn chỉnh cho **Bài 6: Sự cố tài liệu dự án trên GitHub**.

PHẦN 1: BÁO CÁO PHÂN TÍCH NGUYÊN NHÂN & SO SÁNH CỐT LÕI

1. Hai sai lầm cốt lõi trong tư duy của Nam

- **Sai lầm 1: Nhầm lẫn giữa định dạng nhị phân đóng gói (Binary Packaging) và văn bản thuần (Plain Text).**

File `.docx` thực chất là một kho lưu trữ nén định dạng ZIP chứa hàng loạt file XML, font chữ, cấu trúc cây DOM và siêu dữ liệu (metadata). Git vốn là hệ thống quản lý phiên bản theo dòng (*line-by-line*) dành cho văn bản thuần. Khi lưu file `.docx`, Git coi đây là **Binary File (file nhị phân)**. Vì vậy:

    - GitHub không thể render xem trước nội dung trực tiếp trên giao diện web như các file mã nguồn.
    - Khi Nam sửa chỉ một dấu phẩy, toàn bộ gói nén ZIP bị thay đổi mã hash nhị phân, khiến Git không thể tính toán `diff` (thay đổi theo từng dòng) mà bắt buộc ghi đè toàn bộ khối nhị phân. Nếu bạn khác cùng chỉnh sửa, Git không thể tự động hợp nhất (merge) mà báo lỗi **Conflict toàn bộ file**.
- **Sai lầm 2: Đánh đổi tính toàn vẹn dữ liệu lấy hình thức trình bày.**

Nam mang thói quen soạn thảo báo cáo in ấn thông thường vào môi trường lập trình cộng tác. Trong kỹ thuật phần mềm, tài liệu dự án (`README`, `Architecture Guide`) phải đi liền với mã nguồn: nhẹ, đọc được bằng bất kỳ trình soạn thảo thô nào (Terminal, Vim, Nano, VS Code) mà không cần cài bộ phần mềm văn phòng cồng kềnh.

2. Giải thích: Tại sao Markdown an toàn hơn, nhẹ hơn và bẫy mất 1 byte dữ liệu

Bảng đối chiếu bản chất kỹ thuật

Tiêu chí	Microsoft Word (.docx)	Markdown (.md)

Bản chất định dạng	**Rich Text / Binary** (Gói nén ZIP chứa các file XML, quan hệ thư mục, styles)	**Plain Text** (Văn bản thuần mã hóa UTF-8 với các ký tự đánh dấu trực quan)
Dung lượng file	Rất nặng (thường $\ge 20 - 50\text{ KB}$ dù chỉ có vài dòng chữ vì phải chứa metadata)	Cực nhẹ (chỉ vài trăm bytes đến vài KB, đúng bằng dung lượng ký tự thực tế)
Khả năng hiển thị Git Diff	Không hỗ trợ (Git chỉ báo: *"Binary files differ"* hoặc xung đột toàn file)	Hoàn hảo: Git nhận diện chính xác dòng nào thêm (+), dòng nào bớt (-)
Khả năng xem trực tiếp	Không xem được trên Web GitHub, bắt buộc phải tải về máy và mở bằng app	GitHub render trực tiếp thành HTML chuẩn, hiển thị tức thì trên mọi trình duyệt

Phân tích bẫy mất 1 byte dữ liệu (Data Corruption Resilience)

- **Đối với `.docx` (Cấu trúc nén ZIP):** Toàn bộ file phụ thuộc vào bảng chỉ mục `Central Directory` và thuật toán nén `Deflate`. Nếu đường truyền mạng chập chờn làm khuyết chỉ **1 byte** dữ liệu nhị phân hoặc sai lệch checksum:
  - Trình giải nén của hệ điều hành sẽ báo lỗi `Corrupted archive`.
  - Word lập tức chặn mở file với thông báo: *"The file is corrupted and cannot be opened"*, dẫn đến **mất trắng toàn bộ dữ liệu tài liệu**.
- **Đối với `.md` (Plain Text UTF-8):** Mỗi ký tự tồn tại độc lập hoặc theo cụm byte xác định. Nếu mất đúng 1 byte:
  - Tại vị trí lỗi chỉ xuất hiện một ký tự lạ (như ký hiệu ``), toàn bộ các đoạn văn bản, danh sách, công thức và code block trước/sau vị trí đó **vẫn nguyên vẹn và đọc được 100%**.

PHẦN 2: CHUYỂN ĐỔI TÀI LIỆU SANG MARKDOWN CHUẨN (README.md)

Dưới đây là nội dung văn bản giới thiệu dự án "GreenTech" được chuyển hóa từ Word sang cú pháp Markdown tương thích hoàn toàn với StackEdit và GitHub:

Markdown

🌱 GreenTech: Giải Pháp Giám Sát Nông Nghiệp Thông Minh

> Dự án nghiên cứu & ứng dụng IoT phục vụ phát triển nông nghiệp xanh bền vững - Thực hiện bởi sinh viên Học viện Công nghệ Bưu chính Viễn thông (PTIT).

---

1. Giới thiệu dự án

GreenTech là nền tảng quản trị vi khí hậu tự động dành cho nhà kính và vườn ươm thông minh. Hệ thống tự động thu thập thông số môi trường, phân tích dữ liệu theo thời gian thực và kích hoạt các thiết bị chấp hành nhằm tối ưu hóa điều kiện phát triển cho cây trồng, giảm thiểu thất thoát tài nguyên nước.

Các trục giá trị cốt lõi
* **Tự động hóa (Automation):** Giảm 80% công sức can thiệp thủ công của người làm vườn.
* **Tiết kiệm tài nguyên (Resource Efficiency):** Giảm 35% lượng nước tiêu thụ nhờ cơ chế tưới vi lượng chính xác.
* **Độ bền dữ liệu (Reliability):** Sử dụng chuẩn định dạng mở, hạn chế tối đa rủi ro hư hỏng dữ liệu khi truyền tải qua mạng.

---

2. Thông số kỹ thuật & Thiết bị sử dụng

Bảng phân loại phần cứng triển khai trong giai đoạn thử nghiệm:

Thiết bị	Chức năng	Chuẩn giao tiếp	Ghi chú vận hành
ESP32 NodeMCU	SoC xử lý trung tâm và kết nối mạng	Wi-Fi / Bluetooth	Điện áp hoạt động 3.3V
Cảm biến SHT30	Đo nhiệt độ và độ ẩm không khí chính xác cao	I2C	Chống nước tốt hơn DHT11
Module Relay 4 kênh	Đóng/ngắt máy bơm áp lực và quạt đối lưu	Tín hiệu Logic (GPIO)	Cách ly quang Opto an toàn

---

3. Lộ trình thực hiện (Checklist dự án)

Nhóm phát triển theo dõi tiến độ công việc trực tiếp qua danh mục sau:

- [x] Khảo sát thực địa và lập sơ đồ kiến trúc hệ thống
- [x] Thiết kế mạch nguyên lý và hoàn thiện firmware đọc cảm biến
- [ ] Xây dựng giao diện Dashboard theo dõi thời gian thực
- [ ] Kiểm thử tải vận hành liên tục 72 giờ trong môi trường nhà kính
- [ ] Đóng gói bộ tài liệu kỹ thuật chuẩn định dạng Markdown

---

4. Hướng dẫn đóng góp & Quy ước nhóm

Để tránh xung đột (Merge Conflict) trên Git repository của nhóm:
1. **Không** commit các tệp định dạng đóng gói nhị phân như `.docx`, `.rar`, `.zip` vào thư mục tài liệu.
2. Mọi ghi chú kỹ thuật, hướng dẫn cài đặt phải được soạn thảo trực tiếp dưới dạng tệp `.md` (sử dụng các công cụ như StackEdit hoặc VS Code).
3. Mỗi khi chỉnh sửa nội dung, hãy commit theo từng đoạn logic cụ thể kèm thông điệp rõ ràng:
   ```bash
   git add README.md
   git commit -m "docs: cập nhật thông số cảm biến SHT30 trong bảng phần cứng"
   git push origin main
   ```